

Plan inicial de estructura de codigo

Version: 0.1 **Fecha:** 27/01/2026 **Enfoque:** arquitectura modular lista para producto (Lite/Completo).

Este documento propone una estructura de codigo para el espejo con teleprompter de letras sincronizadas: captura de audio o now-playing, identificacion de cancion, obtencion de letra (idealmente con timestamps), motor de sincronizacion y render en modo espejo con interaccion minima.

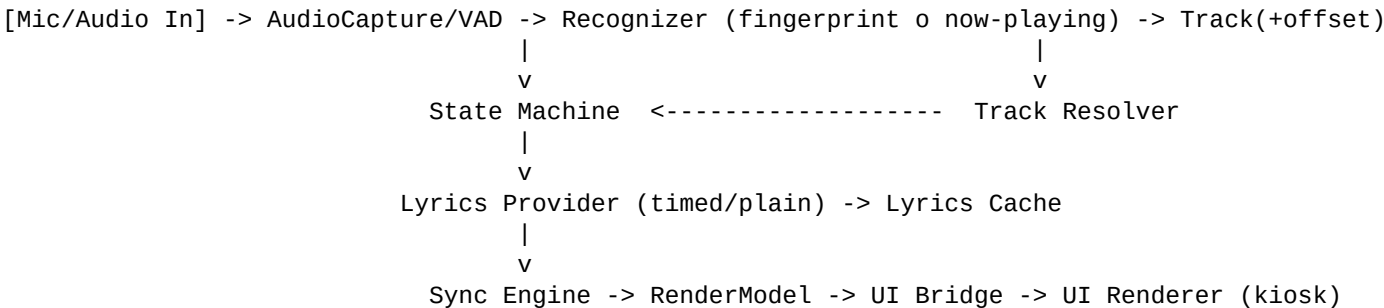
Decisiones clave de arquitectura

- **Monorepo unico** con modulos desacoplados (facil de iterar y versionar).
- **Runtime en 2 procesos:** (1) device-daemon/orquestador y (2) UI kiosk. Comunicacion local por WebSocket o socket UNIX.
- **Interfaces estables** para cambiar proveedores (reconocimiento y letras) sin reescritura.
- **Caches y tolerancia a fallos** desde el inicio (reintentos, backoff, logs y diagnostico).

Por que no un codigo unitario grande

Un unico binario/proceso acelera el prototipo, pero en producto real se vuelve fragil: un crash en UI tumba el core; cambiar proveedor (ACR/Spotify/Musixmatch u otros) es mas costoso; y el soporte requiere logs/telemetria aislada. La separacion UI vs core mejora resiliencia y velocidad de iteracion.

Pipeline de alto nivel



El orquestador (State Machine) gobierna el flujo: detecta musica, identifica el tema, resuelve IDs, pide letra, sincroniza y publica el modelo de render a la UI. La UI solo dibuja.

Modulos recomendados

Modulo	Responsabilidad	Entradas/Salidas principales
libs/audio	Captura de audio, buffer circular, VAD (deteccion de musica)	Audio frames -> chunks normalizados
libs/recognition	Adaptadores de identificacion (cloud fingerprint, now-playing)	Audio chunk/estado -> TrackMatch
libs/lyrics	Proveedores de letras (timed y plain), normalizacion LRC, cache	TrackRef -> TimedLyrics/PlainLyrics
libs/sync	Motor de sincronizacion, correccion de drift, ventana de lineas	posicion_ms + letras -> RenderModel

libs/device	Boton GPIO, brillo/energia si aplica, watchdog	eventos HW -> eventos core
libs/common	Modelos de datos, config, logging, errores	types + utilidades
apps/device_daemon	Orquestacion (state machine), colas, reintentos, cache, telemetria	Publica estado y RenderModel
apps/ui_kiosk	UI en modo espejo: tipografia, contraste, animacion de resaltado	RenderModel -> pantalla

Contratos de datos (interfaces estables)

TrackRef (referencia canonica de cancion)

- Campos sugeridos: provider, provider_track_id, isrc(opcional), title, artist, album(opcional), duration_ms(opcional).
- Usar TrackRef como llave de cache y para resolver letras en distintos proveedores.

TrackMatch (resultado del reconocimiento)

- TrackRef + confidence (0-1) + position_ms (offset estimado) + matched_at (timestamp local).
- Permite corregir drift: si llega un nuevo match con offset distinto, el Sync Engine ajusta suavemente.

TimedLyrics (letra con timestamps)

- Lista de lineas: (start_ms, end_ms opcional, text).
- Normalizacion: limpiar corchetes, etiquetas, duplicados; soportar LRC y formatos equivalentes.
- Fallback: si no hay timed, usar PlainLyrics con heuristica (estimacion por duracion y densidad de texto).

RenderModel (lo que la UI dibuja)

- Ventana de lineas: prev/current/next (por ejemplo 2-1-2) con indice de resaltado.
- Estilo: font_scale, opacity, alignment, mirror_mode=true, theme_id.
- Metadatos: track title/artist, status (listening/identifying/displaying/no_lyrics/error).

Estructura de repositorio (monorepo)

```
espejo-teleprompter/
  apps/
    device_daemon/
    ui_kiosk/
  libs/
    common/
    audio/
    recognition/
    lyrics/
    sync/
    device/
  infra/
    systemd/
    docker/ (opcional)
  tools/
    simulator/
  tests/
```

El monorepo reduce fricción al compartir modelos de datos y utilidades. Los adaptadores (recognition/lyrics) deben ser plug-ins configurables por archivo de config.

State machine del orquestador (device-daemon)

- **IDLE**: espejo normal; pantalla apagada o en standby.
- **LISTENING**: VAD detecta música; bufferiza audio.
- **IDENTIFYING**: llama al recognizer; maneja retries.
- **FETCHING_LYRICS**: resuelve TrackRef y obtiene letras (cache primero).
- **DISPLAYING**: publica RenderModel continuamente.
- **NO_LYRICS**: muestra mensaje y fallback (plain o sugerencia).
- **ERROR**: estado degradado con backoff y logging.

Eventos clave: MusicDetected/MusicStopped, TrackIdentified, LyricsReady, ButtonToggle, NetworkUp/Down. La UI nunca ejecuta lógica de negocio: solo render.

Topología de procesos recomendada

- **Proceso 1 - device-daemon**: captura audio, reconocimiento, obtención/caching de letras, sincronización, estado, logs.
- **Proceso 2 - ui-kiosk**: Chromium/Qt en modo kiosk, espejo (mirror) y tipografía. Reiniciable sin afectar core.

IPC sugerido: WebSocket localhost (fácil para UI web) o socket UNIX (más liviano). Mensajes: RenderModel a 10-20 Hz durante DISPLAYING; estados a 1 Hz en otros modos.

MVP vendible (alcance mínimo)

- Botón on/off (o modo always-on configurable).
- VAD básico para activar pipeline solo con música.
- Un proveedor de reconocimiento (cloud) o now-playing con cuenta vinculada.
- Letras sincronizadas si existen; fallback a letra no sincronizada.
- UI con 2-3 presets: tamaño fuente, opacidad, contraste.
- Cache local: últimas N canciones/letras + TTL para ahorro de API.
- Logs rotativos + pantalla de diagnóstico oculta (para soporte).

Roadmap tecnico sugerido (4 fases)

Fase	Objetivo
Fase 0 - Simulador	Correr pipeline con audio pregrabado. Valida Sync Engine y UI sin hardware final.
Fase 1 - MVP en hardware	Audio real + reconocimiento + letras + UI kiosk. Watchdog y autostart.
Fase 2 - Modelo Completo	Cache robusta, modo degradado sin red, mejoras de drift y latencia.
Fase 3 - Producto	Provisioning, actualizaciones OTA, telemetria opcional, hardening de seguridad, QA de rendim

Riesgos principales y mitigaciones

Riesgo	Mitigacion
Licencias de letras	Definir proveedor/licencia desde temprano; implementar providers intercambiables; cache y
Latencia y offset incorrecto	Buffer y VAD; mostrar estado 'identificando'; refinamiento de offset con re-matches; smoothi
Drift entre audio y timestamps	Correccion gradual; deteccion de saltos; resync automatico al estribillo/segmentos.
Soporte en campo	Logs rotativos, codigos de error, modo 'diagnostico'; procesos separados para reinicio rapid

Proximos pasos recomendados

- Definir stack: core (Python vs Go) y UI (Web kiosk vs Qt) segun hardware objetivo.
- Elegir proveedor inicial de reconocimiento y letras; documentar limites y costos por uso.
- Implementar simulator/ con audio de prueba y conjunto de canciones para QA.
- Implementar contratos TrackRef/TimedLyrics/RenderModel y pruebas unitarias del Sync Engine.
- Integrar systemd (auto-start, restart, watchdog) para estabilidad en demo/field tests.